FinSight AI

Feature Build Guide

How to Build Every Planned Feature

March 2026 | Version 0.1

Introduction

This document provides a detailed technical implementation guide for every feature planned in the FinSight AI platform, organized by the three priority tiers defined in the Project Knowledge Document: Core MVP Features (Priority 1), Enhanced V1 Features (Priority 2), and Aspirational V2+ Features (Priority 3).

Each feature section covers: what it is, how to build it technically, which technologies and APIs to use, the data flow involved, key engineering challenges, and estimated effort. This guide assumes a Python/FastAPI backend, React frontend, and LLM API (Claude or GPT-4o) as the reasoning core.

SECTION 1: Core MVP Features (Priority 1)

These are the foundational features required for the MVP. Without these, the product cannot function. They form the core AI reasoning loop: ingest portfolio → ingest market data → ingest news → generate personalized suggestions.

1.1 Persistent Portfolio Context

What It Is

The system securely ingests and stores a user's complete financial picture — equity holdings, mutual funds, bank balances, fixed deposits, crypto, and real estate — and injects this data into every AI conversation so the AI always knows the user's current financial position.

How to Build It

- Data Ingestion Layer: Build API connectors for Zerodha Kite API, Angel One SmartAPI, and Groww (where available). For banking, use Account Aggregator (AA) framework or Plaid-style open banking APIs. For unsupported brokers, provide CSV/Excel upload + manual entry forms.
- Data Model: Design a normalized PostgreSQL schema with tables for users, portfolios, holdings, transactions, asset_classes, and sync_logs. Each holding row contains: symbol, quantity, average_buy_price, current_price, asset_type, broker_account_id, last_synced_at.
- Real-Time Sync Engine: Use background Celery/APScheduler jobs to call broker APIs every 15-30 minutes. Store delta diffs — only update changed rows — and log each sync event for audit.
- RAG Context Injection: Before each LLM call, query the portfolio DB for the user. Serialize the portfolio into a structured JSON/text summary (top holdings, sector exposure, total value, asset allocation breakdown). Prepend this to the LLM system prompt as context.
- Manual Entry Fallback: Build a form-based UI for manual entry of holdings, FD details (principal, interest rate, maturity), and PPF/EPF balances. These are stored identically in the same schema.

- Security: Encrypt all OAuth tokens at rest using AES-256. Never store raw credentials. Implement token refresh logic. Use row-level security (RLS) in PostgreSQL so users can only query their own data.

Effort	6-8 weeks (backend engineer + ML engineer for context assembly)
Key APIs	Zerodha Kite API, Angel One SmartAPI, CDSL/NSDL AA Framework
Database	PostgreSQL (structured), Redis (caching current prices)
Key Challenge	Broker API rate limits; handling failed syncs gracefully with retry logic

1.2 Real-Time Market Intelligence

What It Is

Live feeds of NSE/BSE equity prices, commodity prices, currency rates, and global indices. The system generates plain-English daily/weekly market summaries specifically relevant to the user's portfolio.

How to Build It

- Market Data Pipeline: Subscribe to Polygon.io (global markets) and NSE India API (Indian equities) via WebSocket for live tick data. For commodities, use Alpha Vantage or MCX API. Store OHLCV (Open, High, Low, Close, Volume) data in a time-series store (TimescaleDB or InfluxDB).
- Sector Mapping: Maintain a static reference table mapping each stock symbol to its SEBI sector and sub-sector (e.g., RELIANCE.NS → Energy → Oil & Gas). This enables the AI to reason at sector level.
- Current Price Cache: Use Redis to cache the latest price for each symbol (TTL: 30 seconds for equities during market hours, 1 hour after hours). Every portfolio value calculation reads from this cache.
- Market Summary Generation: Build a daily cron job that runs at market close (3:30 PM IST). It queries: (a) which symbols the user holds, (b) their price movements today, (c) sector-level index moves. It then calls the LLM with this data + user portfolio to generate a personalized plain-English daily digest.
- Sector-Level Intelligence: Map user holdings to sectors. Monitor sector ETF/index prices (Nifty IT, Nifty Bank, Nifty Auto, etc.) as proxies for sector health. Alert the user when their exposed sectors move significantly.
- Global Indices: Pull data for S&P 500, NASDAQ, FTSE, Nikkei, Hang Seng, Crude Oil (WTI/Brent), Gold, USD/INR. Store as a daily reference time-series.

Effort	3-4 weeks (backend engineer)
Key APIs	Polygon.io, NSE India API, Alpha Vantage, Yahoo Finance (fallback)
Database	TimescaleDB (time-series), Redis (live price cache)

Key Challenge	NSE API rate limits; handling market holiday schedules; data normalization across sources

1.3 Global News Monitoring & Impact Analysis

What It Is

Continuous ingestion of financial and geopolitical news. Each article is scored for relevance to the user's portfolio. The AI maps news events to specific holdings — e.g., 'Rising crude oil prices may impact your ONGC and HPCL positions.'

How to Build It

- News Ingestion: Poll RSS feeds from Reuters, Economic Times, Moneycontrol, Bloomberg, and Mint every 5-10 minutes. Use NewsAPI for broader financial news coverage. Use GDELT for geopolitical event tracking (conflict, sanctions, policy changes).
- Deduplication: Hash the article URL + title. Store in a news_articles table with a unique constraint on the hash. Skip duplicates on insert.
- NLP Preprocessing: For each article, extract: (a) named entities (company names, person names, countries) using spaCy or a fine-tuned NER model, (b) mentioned stock tickers using a ticker-to-company name lookup table, (c) sector tags (Energy, Banking, IT, etc.).
- Relevance Scoring: Build a scoring function that checks if any of the article's extracted tickers or sectors overlap with the user's holdings. Assign a relevance score (0-1). Articles with score > 0.5 are flagged for the user.
- Sentiment Analysis: Use a financial sentiment model (FinBERT, available on HuggingFace) to classify each article as Positive / Negative / Neutral for each mentioned entity. Store sentiment score per entity.
- Personal Impact Mapping: For high-relevance articles, pass the article + user portfolio summary to the LLM. Ask it to explain: 'How might this news event affect this user's specific holdings, and what action, if any, should they consider?' Store this as an insight linked to the user and article.
- Embeddings Store: Embed each article using a sentence transformer model (text-embedding-3-small via OpenAI API, or locally using sentence-transformers). Store in Pinecone/Weaviate. This enables semantic search — 'What recent news is relevant to my HDFC Bank holding?'

Effort	5-7 weeks (ML engineer + backend engineer)
Key APIs/Models	NewsAPI, GDELT, FinBERT (HuggingFace), OpenAI Embeddings API
Database	PostgreSQL (articles), Pinecone/Weaviate (vector embeddings)
Key Challenge	Entity resolution (mapping 'HDFC' vs 'HDFC Bank' vs 'HDFC Ltd' correctly); news deduplication at scale

1.4 Personalized Financial Suggestions

What It Is

The core AI reasoning output: goal-based advice, emergency fund monitoring, rebalancing suggestions, and tax optimization hints — all grounded in the user's actual portfolio and stated financial goals.

How to Build It

- Goal Capture: During onboarding, ask users to define 1-3 financial goals. Each goal captures: goal type (Retirement, Home Purchase, Education, Emergency Fund), target amount (INR), target date, current corpus dedicated to that goal, and risk tolerance (Conservative / Moderate / Aggressive).
- Goal-Based Advice Engine: At suggestion generation time, pull the user's goals from the DB. Calculate: current corpus vs target, years remaining, required monthly SIP to reach goal given assumed return rate (use conservative assumptions: 10-12% CAGR for equity, 7% for debt). Pass this gap analysis to the LLM to frame all advice in goal-context.
- Emergency Fund Monitor: Store the user's declared monthly expense figure. Emergency fund target = 6x monthly expenses. Compare against liquid holdings (savings balance + liquid MFs + short-term FDs). If below threshold, generate an alert with a specific suggestion (e.g., 'Consider moving INR 50,000 from your equity holdings to a liquid fund to rebuild your emergency buffer').
- Rebalancing Suggestions: Calculate the user's current asset allocation (% equity, % debt, % gold, % real estate). Compare to their target allocation (set during onboarding or derived from risk profile). If any asset class is more than 5% off target, generate a rebalancing suggestion specifying which holdings to trim and where to redeploy.
- Tax Optimization: Track each holding's purchase date and purchase price. Calculate LTCG/STCG status (equity LTCG > 1 year; debt LTCG > 3 years). Identify holdings in loss that could be harvested before March 31. Identify ELSS holdings for 80C. Pass this data to the LLM to generate specific tax-saving suggestions. Flag LTCG approaching the INR 1 lakh annual exemption limit.
- Suggestion Ranking and Delivery: All generated suggestions are stored in a suggestions table with fields: user_id, suggestion_text, category, urgency (High/Medium/Low), source_trigger (news/portfolio/goal/tax), generated_at, dismissed_at. The frontend surfaces top 3 undismissed suggestions on the dashboard.

Effort	6-8 weeks (full-stack + ML engineer)
Key Tech	LLM API (Claude/GPT-4o), custom rule engine for LTCG/STCG calculations
Database	PostgreSQL (goals, suggestions, holdings with buy dates)
Key Challenge	Avoiding generic advice — every suggestion must be grounded in specific user data; preventing LLM hallucination on financial figures

SECTION 2: Enhanced V1 Features (Priority 2)

These features are built after the MVP has been validated with initial users. They significantly deepen the product's intelligence and user experience.

2.1 Risk Profile Engine

What It Is

A dynamic risk assessment system that evaluates the user's current risk exposure and adjusts recommendations accordingly. Unlike static questionnaire-based risk profiles, this engine updates based on market volatility, life events, and observed user behavior.

How to Build It

- Initial Questionnaire: Build an onboarding risk questionnaire covering: investment horizon, reaction to portfolio drops (behavioral), income stability, existing liabilities, age and dependents, and existing emergency fund. Score each response and assign a base risk score (1-10).
- Portfolio Risk Score: Calculate quantitative risk metrics from the portfolio: Beta (sensitivity to Nifty 50), standard deviation of returns, Sharpe ratio, maximum drawdown over 1Y and 3Y periods. Weight these into a Portfolio Risk Index (PRI).
- Dynamic Adjustment Triggers: Automatically re-score when: (a) Market VIX crosses 20 (high volatility) — reduce recommended equity exposure, (b) A life event is declared (marriage, child, job loss) — recalculate horizon and income stability, (c) The user makes 3+ rapid trades in a week — flag potential overtrading, (d) The portfolio concentration in a single stock exceeds 20% — flag concentration risk.
- Risk-Return Profiling: Map each holding to a risk tier (Large Cap, Mid Cap, Small Cap, International, Debt, Gold, Real Estate). Calculate the portfolio's weighted average risk tier. Compare to the user's declared risk appetite and flag mismatches.
- XGBoost Risk Model (Optional): Train an XGBoost model on historical user behavior + market data to predict the probability of panic-selling behavior under stress scenarios. Use SHAP values to explain which factors drove a risk score change.

Effort	4-5 weeks (ML engineer)
Key Tech	Python (scipy, numpy for risk metrics), XGBoost (optional ML layer), NSE historical data
Key Challenge	Keeping risk scores explainable — users need to understand why their risk profile changed

2.2 Goal Simulator

What It Is

An interactive tool that projects future wealth scenarios under different assumptions. Users can see what their portfolio will look like in 10, 20, or 30 years with and without following FinSight's suggestions, using Monte Carlo simulation.

How to Build It

- Monte Carlo Simulation Engine: Write a Python function that runs N=1000 simulations of portfolio growth. Each simulation: (a) draws a random annual return for each asset class from a normal distribution (mean = historical average, std = historical volatility), (b) applies SIP contributions, withdrawals, and goal redemptions, (c) calculates portfolio value at each year from now to goal year. Output: 10th, 50th, and 90th percentile portfolio value projections.
- Scenario Comparison: Run two parallel simulations — 'Current Path' (no changes) vs 'Suggested Path' (applying FinSight's rebalancing and SIP suggestions). Display the gap between the two outcomes in a chart.
- Goal Progress Visualization: For each goal, show: probability of achieving the target (% of simulations that reach the target amount by goal date), required SIP to achieve goal with 80% probability, shortfall/surplus at goal date under median scenario.
- Sensitivity Sliders: Build an interactive UI where users can drag sliders to change: monthly SIP amount, expected return rate, and goal date. Re-run simulations in real-time (< 1 second) using a pre-compiled NumPy vectorized simulation function served via FastAPI.
- Inflation Adjustment: Apply India CPI inflation (default: 6% per year) to convert nominal projections to real (purchasing power-adjusted) values. Show both nominal and inflation-adjusted projections.

Effort	3-4 weeks (ML engineer + frontend engineer)
Key Tech	Python (NumPy for vectorized Monte Carlo), React (interactive chart), Recharts or D3.js
Key Challenge	Simulation speed — 1000 simulations x 30 years must run in < 500ms; use NumPy vectorization, not Python loops

2.3 Smart Alerts

What It Is

Intelligent push notifications and in-app alerts triggered by significant events: a portfolio holding drops 5%, an RBI policy decision is announced, an earnings release is coming up, or a macro event directly impacts the user's holdings.

How to Build It

- Alert Rule Engine: Build a rule-based alert engine in the backend. Alert rules are defined as: (trigger_type, condition, threshold, user_scope). Examples: PRICE_DROP — stock price falls > 5% in one session; EARNINGS_UPCOMING — earnings date within 3 days for a held stock; NEWS_IMPACT — a news article scores > 0.8 relevance for the user's portfolio; RBI_EVENT — RBI rate decision detected in news feed.

- Event Detection: A background Celery worker runs every 5 minutes. It checks: latest prices vs session open for price drop rules; an events calendar API (earnings dates from NSE API or Tickertape) for corporate events; the news pipeline for high-impact articles.
- Alert Deduplication: Store alert history in an alerts table. Before sending any alert, check if an identical alert (same user, same trigger, same holding) was sent in the last 24 hours. If yes, suppress.
- Push Notification Delivery: Use Firebase Cloud Messaging (FCM) for mobile push notifications. Store FCM device tokens per user. For web, use Web Push API (VAPID keys). Fallback to email via SendGrid for critical alerts.
- AI-Enhanced Alert Text: Don't send raw alerts like 'INFY dropped 4.2%'. Pass the event to the LLM with portfolio context to generate: 'Your Infosys holding (₹1.2L, 8% of portfolio) dropped 4.2% today following a weak Q3 guidance. This is likely sector-wide — Nifty IT is also down 3.1%. No immediate action required unless your outlook on IT has changed.'

Effort	3-4 weeks (backend engineer + ML engineer)
Key Tech	Celery (background jobs), FCM (push), SendGrid (email), LLM API for alert text
Key Challenge	Avoiding alert fatigue — tuning thresholds so users get important alerts but not noise

2.4 Portfolio Stress Testing

What It Is

A simulation that shows how the user's current portfolio would have performed under historical crisis scenarios: the 2008 Global Financial Crisis, COVID-19 crash (March 2020), 2013 Taper Tantrum, and 2022 rate hike cycle.

How to Build It

- Historical Scenario Library: Curate a library of stress scenarios. For each scenario, define: scenario_name, start_date, end_date, peak_to_trough_dates, and sector-level drawdown percentages (e.g., in COVID crash: Large Cap Equity -38%, Mid Cap -45%, Gold +12%, Nifty IT -30%, Nifty Bank -40%).
- Stress Test Engine: For a given user portfolio, for each scenario: (a) map each holding to its asset class and sector, (b) apply the historical drawdown % for that sector in that scenario to the holding's current value, (c) aggregate to calculate portfolio-level drawdown, (d) calculate recovery time (how long it took markets to recover to pre-crisis levels).
- Recovery Analysis: Show not just the drawdown but the recovery timeline. E.g., 'Under a COVID-like scenario, your portfolio would drop to ₹X (from ₹Y). Markets recovered to pre-crash levels in 14 months. Your equity-heavy allocation would recover faster than a debt-heavy portfolio.'

- Actionable Output: After the stress test, pass results to the LLM: 'Given this stress test result, what specific changes to the user's current portfolio would make it more resilient?' Generate 2-3 concrete suggestions.
- Comparison Mode: Allow the user to run stress tests on two portfolio versions — current vs. rebalanced — to visually see how diversification improves resilience.

Effort	2-3 weeks (ML engineer)
Key Tech	Python (NumPy), historical market data from NSE/BSE or Bloomberg data dumps
Key Challenge	Accurately mapping holdings to historical price series for older/delisted stocks; data availability for Indian mid-cap stocks pre-2010

2.5 Competitor Comparison (Benchmarking)

What It Is

Compares the user's portfolio performance against relevant benchmarks: Nifty 50, Nifty Midcap 150, Nifty Small Cap 250, and relevant mutual fund category averages. Helps users objectively assess whether their portfolio is outperforming or underperforming.

How to Build It

- Benchmark Index Data: Pull index historical price data from NSE (free) or Polygon.io. Maintain a benchmark_prices table with daily closing values for Nifty 50, Nifty Midcap 150, Nifty Small Cap, Nifty 500, Sensex, and 3-4 key sectoral indices.
- XIRR Calculation: Calculate the user's actual XIRR (Extended Internal Rate of Return) using the actual transaction history (buy dates, buy prices, quantities, dividends received). Compare this XIRR to the benchmark CAGR over the same period. This is the most meaningful comparison.
- Alpha Calculation: Compute portfolio alpha = user's XIRR - benchmark CAGR over 1Y, 3Y, and 5Y periods. Display whether the user is generating alpha (outperforming) or negative alpha (underperforming) relative to passive investing.
- Mutual Fund Category Comparison: Use AMFI public data (free) to fetch NAV history for mutual fund categories. Compare the user's MF holdings against their category average. E.g., 'Your Large Cap MF has returned 14.2% CAGR vs category average of 13.8% — top quartile.'
- Contextual Interpretation: Pass the benchmark comparison data to the LLM to generate a plain-English assessment: 'Your portfolio has underperformed the Nifty 50 by 2.3% over the last year, primarily due to your mid-cap exposure during the correction. However, your 3-year XIRR of 17.4% still beats the index's 14.2%.'

Effort	2-3 weeks (backend engineer)
Key Tech	NSE API (index data), AMFI (MF NAV data), scipy (XIRR calculation)

Key Challenge	XIRR calculation requires complete transaction history — users who manually entered data without full transaction history will get approximate results

SECTION 3: Aspirational V2+ Features (Priority 3)

These features represent the long-term vision of FinSight AI. They require a more mature product, user base, and engineering team. Building them before the MVP is validated would be premature.

3.1 Voice Interface

What It Is

Conversational portfolio queries via voice: 'How did my portfolio do this week?', 'What's my HDFC Bank exposure?', 'Should I add more to my SIP this month?'

How to Build It

- Speech-to-Text (STT): Use OpenAI Whisper API (cloud) or a locally hosted Whisper model for transcription. Whisper handles Indian English accents well and supports Hindi-English code-switching, which is important for the Indian market.
- Intent Parsing: After transcription, pass the text query through the existing LLM pipeline — the same AI reasoning engine used for text chat. No separate NLU (Natural Language Understanding) model is needed; the LLM handles intent classification natively.
- Text-to-Speech (TTS): Use ElevenLabs API or Google Cloud TTS for response audio. For Indian English, Google Cloud TTS has better accent support. Cache frequently generated audio responses (e.g., market summaries) to reduce API latency.
- Mobile Integration: In the React Native app, use the device microphone via expo-av (Expo) or react-native-audio-recorder. Stream audio chunks to the backend via WebSocket for low-latency transcription. Play back the TTS response via the device speaker.
- Wake Word Detection (Optional): For a truly hands-free experience, integrate a wake word model ('Hey FinSight') using Picovoice Porcupine (on-device, privacy-preserving). This is purely optional for V2.

Effort	4-5 weeks (mobile engineer + backend)
Key APIs	OpenAI Whisper (STT), ElevenLabs or Google TTS, Picovoice (optional wake word)
Key Challenge	Latency — voice responses must feel instantaneous; stream TTS audio as it's generated rather than waiting for the full response

3.2 Family Financial Hub

What It Is

A household-level view that consolidates the financial portfolios of all family members (spouse, parents, children) under one primary account. Useful for families where one person manages finances for multiple members.

How to Build It

- Multi-Profile Data Model: Extend the data model to support a household entity. A household has one primary_user_id and a list of member_user_ids. Each member has their own portfolio, goals, and risk profile but shares household-level reporting.
- Invitation Flow: The primary user sends an invitation (by email/phone) to family members. Family members sign up independently and grant explicit consent to share their portfolio data with the household primary user. This consent is logged for DPDP Act compliance.
- Household Dashboard: Aggregate all member portfolios into a combined household view. Show: total household net worth, asset allocation across all members, goal coverage (which goals are fully/partially funded), household liquidity (combined liquid assets).
- Tax Optimization Across Family: Enable family-level tax planning — e.g., gift of equity to spouse/parents in lower tax bracket, combining LTCG exemptions, coordinating 80C investments across family members.
- Access Control: The primary user can view all member portfolios. Members can only see their own data. The primary user cannot make trades or suggestions on behalf of members without explicit member approval (a consent-gated action).

Effort	5-6 weeks (full-stack engineer)
Key Tech	PostgreSQL (household schema), OAuth 2.0 (per-member auth), DPDP-compliant consent logging
Key Challenge	Consent management — every data-sharing relationship must be logged, auditable, and revocable. This is a significant legal and engineering surface area.

3.3 Document Intelligence

What It Is

Users can upload financial documents — SIP statements, ITR filings, insurance policies, salary slips, Form 16 — and the AI extracts and analyzes the information, adding it to the user's financial picture.

How to Build It

- Document Ingestion: Accept PDF, JPEG, PNG uploads via the frontend. Pass to a document parsing pipeline: (a) for text-based PDFs, use pdfplumber or PyMuPDF to

extract text, (b) for scanned documents/images, use Google Document AI or AWS Textract for OCR.

- Document Classification: Build a classifier (fine-tuned BERT or a simple prompt-based LLM classifier) that identifies document type: SIP Statement, ITR, Form 16, Insurance Policy, Salary Slip, Broker Contract Note.
- Structured Extraction: For each document type, define an extraction schema. E.g., for a SIP Statement: {fund_name, folio_number, sip_amount, sip_date, total_units, current_nav, current_value}. Use the LLM with a structured output prompt (JSON mode) to extract these fields from the parsed text.
- Portfolio Reconciliation: Compare extracted holdings against existing portfolio DB records. Identify discrepancies (e.g., SIP statement shows 5,000 units of Axis Bluechip but the DB only has 4,800 — flag for user review).
- Tax Document Processing: For ITR and Form 16, extract: total income, taxable income, 80C investments made, TDS deducted. Use this to update the tax optimization engine with accurate figures for the current financial year.
- Insurance Gap Analysis: Extract insurance policy details: sum assured, premium, policy term, coverage type. Pass to the LLM with the user's portfolio and liabilities to assess whether coverage is adequate relative to the user's financial exposure.

Effort	6-8 weeks (ML engineer + backend engineer)
Key Tech	AWS Textract or Google Document AI (OCR), pdfplumber (text PDF), LLM (structured extraction)
Key Challenge	Indian financial document formats are inconsistent across brokers and banks — extraction rules need to handle high variability. Build a feedback mechanism so users can correct extraction errors.

3.4 Behavioral Coaching

What It Is

Detects behavioral patterns — panic selling, overtrading, recency bias, concentration bias — and provides nudges grounded in behavioral finance principles to help users make more rational decisions.

How to Build It

- Behavioral Pattern Detection: Analyze the user's transaction history for patterns: (a) Panic Sell Signal: sells > 20% of equity holdings within 2 trading sessions of a market drop > 5%. (b) Overtrading Signal: more than 10 transactions in a rolling 30-day window with no clear rebalancing rationale. (c) Recency Bias Signal: buys heavily into an asset that has risen > 30% in the last 3 months. (d) Home Country/Sector Bias: > 80% exposure to a single sector or all Indian equities with no international diversification.
- Behavioral Score: Maintain a Behavioral Rationality Score (1-10) per user. It decreases when problematic patterns are detected and increases over time with disciplined behavior (consistent SIPs, maintaining allocation targets).

- Intervention Nudges: When a pattern is detected, trigger a behavioral nudge — not a stern warning, but a thoughtful prompt. Pass the detected pattern to the LLM with behavioral finance context to generate an empathetic, evidence-based message. E.g., for panic selling: 'I noticed you're considering selling your equity holdings after today's dip. Historically, markets have recovered within X months of similar drops. Investors who stayed invested during the COVID crash saw 40% gains by December 2020. Would you like to talk through your options before acting?'
- Pre-Trade Friction: Before a user executes a trade that matches a problematic pattern, insert a 'cooling off' step — show the behavioral nudge and ask them to confirm after reading it. This is the behavioral economics principle of 'friction' to reduce impulsive decisions.
- Behavioral Finance Knowledge Base: Build a curated vector database of behavioral finance insights (from Kahneman, Thaler, and DALBAR research). Use RAG to fetch relevant insights when generating nudges — grounds the nudge in cited research rather than generic AI commentary.

Effort	4-5 weeks (ML engineer + product/UX)
Key Tech	Python (pattern detection on transaction data), LLM API (nudge generation), Pinecone (behavioral finance knowledge base)
Key Challenge	Tone calibration — nudges must be empathetic and informative, never preachy or condescending. Requires extensive UX testing.

3.5 Proprietary Fine-Tuned Financial LLM

What It Is

A domain-specific large language model fine-tuned on Indian financial data: SEBI filings, NSE/BSE earnings call transcripts, RBI policy communications, AMFI data, and regulatory circulars. Intended to outperform general-purpose LLMs on Indian market-specific reasoning tasks.

How to Build It

- Base Model Selection: Start with Llama 3 70B (Meta, open weights) or Mistral Large as the base. These are powerful enough to benefit from fine-tuning and can be self-hosted post fine-tuning to avoid ongoing API costs at scale.
- Training Data Curation: Assemble a corpus of Indian financial text data: (a) SEBI filings and circulars (publicly available at sebi.gov.in), (b) NSE and BSE earnings call transcripts (scrapeable or via third-party data providers), (c) RBI monetary policy statements and annual reports, (d) AMFI mutual fund factsheets and scheme information documents, (e) Financial news articles (Economic Times, Moneycontrol, Mint) with permission or Fair Use framing, (f) Indian tax law documents (Income Tax Act sections, LTCG/STCG rules, 80C schedules). Total corpus target: 10-50GB of cleaned text.
- Data Cleaning Pipeline: Build a pipeline using Python + Apache Spark (or Dask for smaller scale) to: strip HTML/PDF boilerplate, deduplicate at sentence level (MinHash LSH), filter out non-financial content, normalize dates and INR figures to consistent formats.

- Fine-Tuning Infrastructure: Use AWS SageMaker or GCP Vertex AI with A100 80GB GPU instances. Fine-tuning a 70B model requires at least 4x A100 GPUs (40-80GB VRAM each) using QLoRA (Quantized LoRA) — a memory-efficient fine-tuning technique. Estimated compute: $20,000-$100,000 depending on dataset size and number of epochs.
- Fine-Tuning Technique: Use LoRA (Low-Rank Adaptation) via the PEFT library (HuggingFace). Fine-tune on instruction-response pairs specific to Indian financial tasks: question-answer pairs from SEBI FAQs, RBI guidelines, and curated financial reasoning examples.
- Evaluation: Build an evaluation benchmark with 200-500 gold-standard question-answer pairs about Indian finance. Compare the fine-tuned model against the base model and GPT-4o on: factual accuracy (LTCG rules, SEBI regulations), reasoning quality on portfolio scenarios, and hallucination rate on Indian-specific financial terminology.
- Deployment: Deploy via vLLM (high-throughput LLM inference server) on AWS EC2 A10G or A100 instances. Expose via the same FastAPI interface used for external LLM APIs — the rest of the application stack needs no changes when switching from Claude API to the self-hosted model.

Effort	4-6 months (ML engineer with LLM training experience)
Cost	$30,000-$130,000 in GPU compute + data licensing (if any)
Key Tech	Llama 3 70B, QLoRA/LoRA (HuggingFace PEFT), AWS SageMaker, vLLM inference server
Key Challenge	Training data quality and legal rights — financial news content may require licensing. Start with purely public domain data (SEBI, NSE, RBI official publications) before adding licensed content.
Pre-Requisite	This feature should only be pursued after MVP validation confirms that a better Indian finance model would meaningfully improve product outcomes.

Implementation Priority & Effort Summary

Feature	Priority	Effort	Key Dependency
Persistent Portfolio Context	**MVP (P1)**	6-8 weeks	Broker APIs
Real-Time Market Intelligence	**MVP (P1)**	3-4 weeks	Polygon.io / NSE API
News Monitoring & Impact Analysis	**MVP (P1)**	5-7 weeks	NewsAPI, FinBERT, Pinecone
Personalized Financial Suggestions	**MVP (P1)**	6-8 weeks	LLM API, Goals DB
Risk Profile Engine	**V1 (P2)**	4-5 weeks	Historical market data
Goal Simulator	**V1 (P2)**	3-4 weeks	Goals DB, NumPy

Feature	Priority	Effort	Key Dependency
Smart Alerts	**V1 (P2)**	3-4 weeks	Celery, FCM
Portfolio Stress Testing	**V1 (P2)**	2-3 weeks	Historical crisis data
Competitor Comparison	**V1 (P2)**	2-3 weeks	NSE index data, AMFI
Voice Interface	**V2+ (P3)**	4-5 weeks	Whisper API, TTS API
Family Financial Hub	**V2+ (P3)**	5-6 weeks	DPDP consent framework
Document Intelligence	**V2+ (P3)**	6-8 weeks	AWS Textract / GDoc AI
Behavioral Coaching	**V2+ (P3)**	4-5 weeks	Transaction history, UX
Proprietary Fine-Tuned LLM	**V2+ (P3)**	4-6 months	GPU budget, training data

Note: *All effort estimates assume a team of 2-3 engineers (1 backend, 1 ML/data engineer, 1 frontend). Solo founder estimates should be roughly 2-3x these timelines. Regulatory research (SEBI RIA compliance) should run in parallel from Day 1 and is not included in these estimates.*